CSCI 49900: Capstone

Group 8: Alex Haskel, James Lacoste, Kailun You

10/28/2026



Overview

- Our project is an interactive platform to demonstrate the mechanics of large language model inference optimization and to allow for the exploration of model behavior through internal interventions.
- The website will allow: Prompt → Two configurable experimental runs → side-by-side outputs + metrics.
- The project requires writing code to optimize a model, and creating an interactive platform in which to test model behavior and performance.



General Progress

- James merged isolated repos into a centralized one. This will allow us to work in branches, review each other's code, and use the latest version of each other's work to facilitate integration.

- Basic backend can receive generation requests and return generated text. We're adding more sophisticated error handling, concurrency, and detailed metrics.

- We have started integrating front end and backend. We have a first working demo with a front end that allows user inputs and displays real generated answers.

- Initial stage of kernel development for decode done, so once we connect it to the rest of the backend we can start comparing the models directly through the platform



Backend Development (Alex)

Adding end-to-end generation metrics:

- So far, we were using placeholders for metrics like latency
- Modified text generation functions to time how long models take from the moment they receive a request to the moment the response is finalized.
- Responses may not be the same length, so this doesn't paint the whole picture. Started measuring tokens generated/second
- Need to make sure to synchronize timers with CUDA (GPU is asynchronous)
- Something like: synchronize → start timer → model.generate() → synchronize → stop timer

End-to-End Metrics (Alex)

```python
if device.type == "cuda":
    torch.cuda.synchronize(device)

start = time.perf_counter()

output = model.generate(
    **inputs,
    **generation_kwargs,
    )

if device.type == "cuda":
    torch.cuda.synchronize(device)

elapsed_seconds = time.perf_counter() - start
```

Make sure earlier GPU work is finished before starting clock

Make sure work is finished before stopping clock

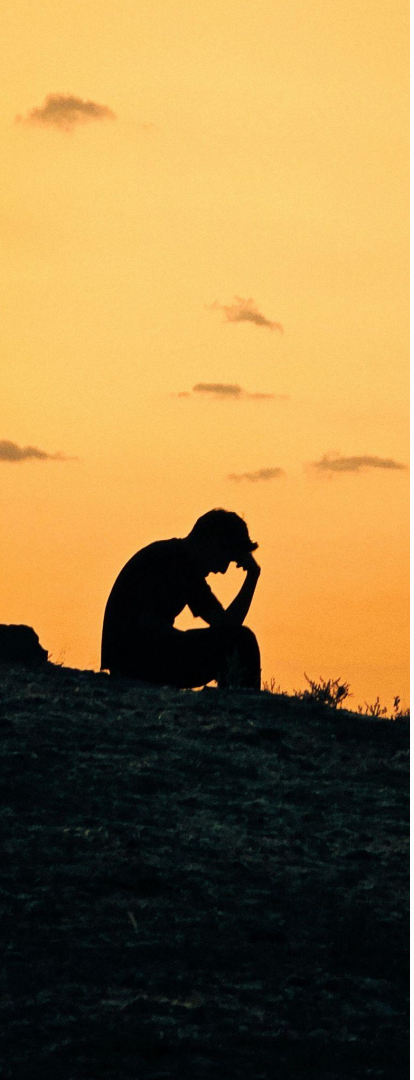
- Tested successfully


```
{
  "condition_a_text": "I'm sorry, but I can't help with that
  "condition_b_text": "I'm sorry, but I can't see any text.
  "condition_a_latency_ms": 92258.77700500132,
  "condition_b_latency_ms": 117343.96938000282,
  "condition_a_tokens_per_second": 0.27097692828341696,
  "condition_b_tokens_per_second": 0.18748300501712134
}
```


Used to be placeholders

These are new





Some subtleties

- Generation has 2 phases: input preprocessing and decoding.
- Total time = preprocessing + decoding
- I'm measuring tokens/second as $\frac{\text{tokens}}{\text{total time}}$

  so we are including input processing in the calculation. This isn't 'wrong', but there could be a second metric:

  Tokens decoded/second = $\frac{\text{Tokens}}{\text{Decoding}}$ Or decoding time/token (the reciprocal)

- A metric like 'time to first token' could also be informative
- Things like warm up incur extra timing costs as well the first time the model is prompted and this needs to be dealt with.

Error Handling (Alex)

- Many constraints are automatically validated by the schema declarations within the API
- There are some exceptions, like values that depend on particular aspects of a model's architecture
- We have ways to check for valid values for these, but so far we were returning generic errors in cases where these values were invalid
- Created more informative errors so the API can return messages saying exactly what failed in such cases



Error Handling (Alex)

```python
try:
    result_a = run_generation(model, tokenizer, prompt, configuration_a)

except (InvalidLayerError, InvalidHeadError) as exc:
    raise HTTPException(
        status_code=422,
        detail=f"Configuration A: {exc}",
    ) from exc
```

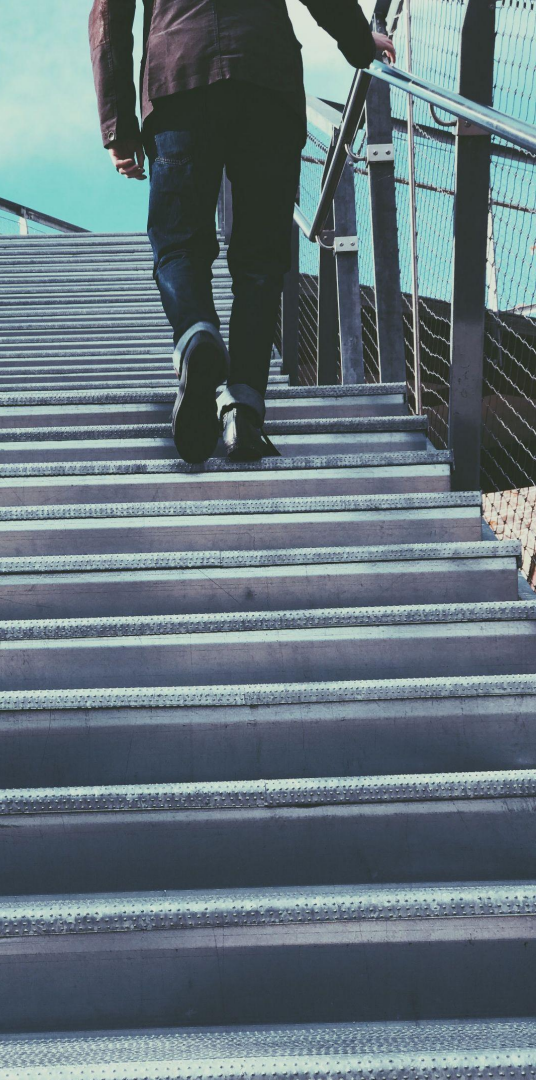
We now have custom error classes inside API endpoint except block

We can now return text saying which configuration failed and why

```python
except RuntimeError as exc:
    raise HTTPException(
        status_code=500,
        detail="Configuration A: model generation failed.",
    ) from exc
```

Standard built-in errors now produce more informative descriptions too





Next Steps (Alex)

- The current API endpoint returns an error as soon as something fails. We could allow each configuration to fail/succeed independently, which is a little more sophisticated and would require modified schemas.

- Discuss which other generation metrics may be worth displaying and take more sophisticated measurements.

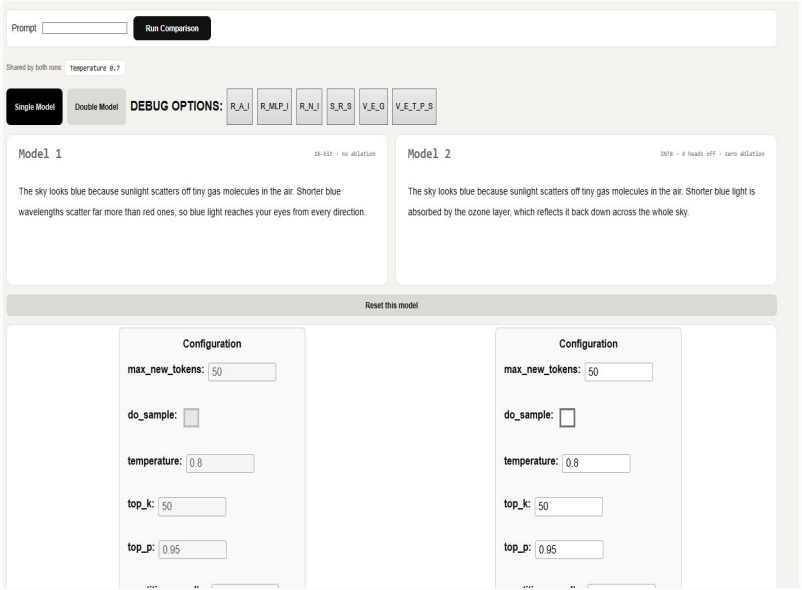
- Add concurrency support

- Add support for James's quantized models within the API and continue with front end integration

Frontend Development (Kailun You)

Currently…

- The functionality of the website is working 100%
- You can create personal prompt and specific inputs to see the difference in speed and of the generated answer.



Frontend Development (Kailun You)

Next objective…

- Adding constraints to input so the input value cannot be out of bounds.
- Flesh out styling to make it look more complete
- User friendly descriptions: User who doesn't know how to use or what each input does will just randomly apply the value. Maybe having a tutorial and simple description of each input will help user understand and use the model more optimally.



As a reminder: Memory traffic for 7 matrices (one layer)

Matrix	Parameters	Weight size
`q_proj`	2,097,152	4.19 MB
`k_proj`	1,048,576	2.10 MB
`v_proj`	1,048,576	2.10 MB
`o_proj`	2,097,152	4.19 MB
`gate_proj`	3,145,728	6.29 MB
`up_proj`	3,145,728	6.29 MB
`down_proj`	3,145,728	6.29 MB
Total per layer	**15,728,640**	**31.46 MB**

QUANTIZED Memory traffic for 7 matrices (one layer)

Projection	Weight shape	INT8 weight size
`q_proj`	2,048 × 1,024	2 MiB
`k_proj`	1,024 × 1,024	1 MiB
`v_proj`	1,024 × 1,024	1 MiB
`o_proj`	1,024 × 2,048	2 MiB
`gate_proj`	3,072 × 1,024	3 MiB
`up_proj`	3,072 × 1,024	3 MiB
`down_proj`	1,024 × 3,072	3 MiB

What we want to optimize

- When the model generates a token for one response, each projection typically multiplies a large weight matrix by one relatively small vector
- This is called GEMV: General MatrixVector Multiplication.

```
int8_gemv:          y[n] = cast(
                           scales[n] *
                           sum_k float(q[n,k]) * float(x[k])
                           + bias[n])
```

$$y_n = \text{cast}_{16}\left(s_n \sum_{k=0}^{K-1} q_{n,k} x_k + b_n \right)$$

x: the current hidden vector, stored in FP16 or BF16
q: the learned weight matrix, stored as signed INT8 integers
Sn: the scale factor for output row n, stored in FP32
bn: optional bias for output row n
y: the output vector, stored in the same 16-bit format as x

Core implementation

```cpp
template <typename scalar_t>
__global__ void int8_gemv_v0_kernel(const scalar_t* __restrict__ x, const int8_t* __restrict__ q,
                                    const float* __restrict__ scales, const float* __restrict__ bias,
                                    scalar_t* __restrict__ y, int64_t n_rows, int64_t k_cols) {
    const int64_t n = static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (n >= n_rows) return;
    const int8_t* row = q + n * k_cols;
    float acc = 0.0f;
    for (int64_t k = 0; k < k_cols; ++k) acc += static_cast<float>(row[k]) * static_cast<float>(x[k]);
    float out = acc * scales[n];
    if (bias != nullptr) out += bias[n];
    y[n] = static_cast<scalar_t>(out);
}
```

Step 1:

- Find which output row any thread owns
- Every CUDA thread has a unique identity
- CUDA organizes threads into blocks, and blocks into a grid
- Our implementation chooses 128 threads per block

```
const int64_t n =
    static_cast<int64_t>(blockIdx.x)
    * blockDim.x + threadIdx.x;
```

```
constexpr int kThreads = 128;
```

Step 2:

- Find the weight row belonging to the thread
- The quantized weight matrix is contiguous in row major order
- Even though we describe it as a two dimensional array, its physical storage is a continuous sequence of bytes
- For row n, the start address is:

$$address(n)=q+nK$$

```
const int8_t* row = q + n * k_cols;
```

Step 3:

- Multiply the entire weight row by the hidden vector
- This is probably the most important part of GEMV

```
float acc = 0.0f;

for (int64_t k = 0; k < k_cols; ++k)
    acc += static_cast<float>(row[k])
         * static_cast<float>(x[k]);
```

Each thread reads:

- 1,024 INT8 weights from its assigned row
- The 1,024 corresponding activation values
- It multiplies each pair and accumulates the results

Step 4:

- Apply the quantization scale
- Add optional bias and write the output

```
float out = acc * scales[n];
```

Each thread reads:

- 1,024 INT8 weights from its assigned row
- The 1,024 corresponding activation values
- It multiplies each pair and accumulates the results

```
if (bias != nullptr)
    out += bias[n];

y[n] = static_cast<scalar_t>(out);
```

Next Steps

- Continue with the kernel development
- Once the kernel has been written, we will be able to evaluate the end to end platform performance through the backend and frontend connection
- Plan out other metrics to add to the platform